

VirtuEx 성능 개선 감사 보고서 (7차)

VirtuEx Performance Improvement Audit Report (7th)

- 감사일: 2026-03-18
- 감사 범위: 프론트엔드 렌더링/Memoization, 서버 페이지네이션, 번들 최적화, 이미지 처리, WebSocket 최적화, DB 쿼리 패턴, 캐싱 전략
- 감사 방법: 소스 코드 정적 분석, 렌더링 패턴 추적, 쿼리 패턴 추적, 번들/캐싱 전략 검토
- 심각도 기준: Critical(즉시 수정) / High(1주 내) / Medium(2주 내) / Low(개선 권장)
- 비고: 발견 사항 정리만 수행하며, 수정은 포함하지 않음

이전 감사 대비 수정 현황 (6차 → 7차)

이전 이슈	상태	비고
[FR-H-01] AssetList useMemo 의존성 과다	부분 수정	assetsRef + assetsVersion 도입으로 참조 안정성 개선. 단, assetsVersion useMemo 자체가 매 tick 실행됨 (하단 PF-M-04 참조)
[FR-H-02] 리더보드 FLIP 애니메이션 리플로우	수정 됨	변경된 행만 선별 애니메이션 적용 (movedEls 배치 FLIP), will-change: transform 적용
[PF-M-01] 커뮤니티 페이지 단일 컴포넌트 과다 상태	미수정	944줄 단일 컴포넌트 유지 (하단 PF-M-01 재확인)
[API-M-01] 주문 서버 페이지네이션 미사용	부분 수정	백엔드 offset/limit 파라미터 지원 확인됨, 프론트엔드에서 미활용 (하단 API-M-01 참조)
[API-M-02] 체결내역 서버 페이지네이션 미사용	부분 수정	백엔드 지원 확인됨, 프론트엔드에서 전체 조회 후 클라이언트 필터 (하단 API-M-02 참조)
[IMG-M-01] TipTap Base64 이미지	미수정	여전히 readAsDataURL로 Base64 인라인 저장 (하단 IMG-M-01 참조)
[BD-L-01] lucide 아이콘 re-export	미수정	트리셰이킹으로 실질 영향 최소, 보류
[WS-L-01] 채팅방 폴링 빈도	수정 됨	useChatRooms refetchInterval 30000ms로 적절히 설정

요약 (Executive Summary)

카테고리	Critical	High	Medium	Low	합계
A. 프론트엔드 렌더링/Memoization	0	1	3	1	5
B. API/서버 페이지네이션	0	1	2	0	3
C. 이미지/에셋 처리	0	0	1	1	2
D. 번들/코드 분할	0	0	1	1	2
E. DB 쿼리 패턴	0	0	1	1	2

F. 캐싱/메모리 관리	0	0	1	1	2
G. WebSocket/실시간 처리	0	0	0	1	1
합계	0	2	9	6	17

A. 프론트엔드 렌더링/Memoization — 5건

[PF-M-01] 커뮤니티 페이지 — 944줄 단일 컴포넌트 미분할 (Medium, 미수정)

현상: `community/page.tsx` 가 944줄의 단일 컴포넌트로 4개 탭(자유게시판, 전략 공유, 트레이더 랭킹, 활동 피드)의 모든 상태와 로직을 포함. 22개 이상의 `useState` 와 12개의 `useMemo/useCallback` , 8개의 `useEffect` 호출이 모든 탭에서 동시에 활성화되어 불필요한 데이터 fetching과 메모리 점유 발생 **위치:** `frontend/src/app/(main)/community/page.tsx:1-944` **영향:** 탭 전환 시 불필요한 리페치, 초기 로드 시 모든 `useEffect` 호출 초기화 **권장:** 탭별로 별도 컴포넌트 분리 + `React.lazy` 또는 조건부 마운트로 비활성 탭 `useEffect` 실행 방지 **심각도:** Medium

[PF-M-04] AssetList assetsVersion useMemo — 매 tick 실행 (Medium)

현상: `AssetList.tsx:101-108` 의 `assetsVersion` `useMemo`가 `assets.length` 와 `currentSymbols` 를 의존성으로 가지지만, `currentSymbols` 자체가 `assets` 배열로부터 매번 재계산됨. `assets` 참조가 `WebSocket` tick마다 변경되므로 `currentSymbols` 도 매번 새로 생성되어 문자열 비교(`.join(',')`)가 매 렌더마다 실행 **위치:** `frontend/src/components/market/AssetList.tsx:98-108` **영향:** 100개+ 심볼 문자열 join + 비교가 500ms마다 발생 (`WebSocket` 배치 주기) **권장:** `currentSymbols` 계산을 `assets` 참조가 아닌 심볼 Set 해시 기반으로 변경하거나, 상위 컴포넌트에서 심볼 목록을 안정적 ref로 전달 **심각도:** Medium

[PF-M-05] 주문 페이지 — trades 전체 로드 후 분석 탭에서 O(N^2) 순회 (Medium)

현상: `orders/page.tsx:186-193` 의 `AnalysisTab`에서 `stats.breakdown` 계산 시 각 symbol에 대해 `trades.filter()` 2회 호출(`buyCount`, `sellCount`). N개 심볼 x M개 거래에서 O(N*M) 복잡도 **위치:** `frontend/src/app/(main)/orders/page.tsx:186-193` **영향:** 거래 내역 1000건+ 시 분석 탭 전환 지연 **권장:** `symbolMap` 순회 시 `buyCount/sellCount`를 함께 집계하여 단일 순회로 통합 **심각도:** Medium

[PF-H-01] 대시보드 AI 분석 — 뉴스 200건 전체 fetch 후 50건 슬라이싱 (High)

현상: `dashboard/page.tsx:178` 에서 AI 분석 시 `api.get('/api/news', { params: { limit: 200 } })` 로 200건 전체를 가져온 후 24시간 내 50건만 사용. 동일 패턴이 `news/page.tsx:157` 에서도 반복 **위치:** `frontend/src/app/(main)/dashboard/page.tsx:178` , `frontend/src/app/(main)/news/page.tsx:157` **영향:** 불필요한 네트워크 전송량 (최대 4배 과잉), 서버 DB 쿼리 부하 **권장:** 서버에서 `publishedAfter` 파라미터로 24시간 내 뉴스만 조회하거나, `limit`을 50으로 줄이고 서버 측 날짜 필터 적용 **심각도:** High

[PF-L-01] PortfolioHistoryChart — 매 렌더마다 Date 객체 다수 생성 (Low)

현상: `PortfolioHistoryChart.tsx:51-53` 에서 `buildHistoryFromTransactions` 함수가 호출될 때마다 전체 거래 내역을 정렬하고 각 거래에 `new Date()` 생성. `totalValue` prop 변경 시(`WebSocket` 가격 업데이트) 매번 재계산 **위치:** `frontend/src/components/portfolio/PortfolioHistoryChart.tsx:51-107` **영향:** 거래 내역 500건+ 시 잦은 GC 발생 가능 **권장:** `totalValue` 변경 시에는 마지막 데이터 포인트만 업데이트하도록 최적화 **심각도:** Low

B. API/서버 페이지네이션 — 3건

[API-M-01] 주문 내역 — 서버 페이지네이션 미사용 (High, 미수정)

현상: useOrders 혹은 (hooks/useOrders.ts:41-78)이 서버에 limit / offset 파라미터를 전달하지 않고 전체 주문을 가져옴. 백엔드 OrderController.getUserOrders() 는 limit/offset을 지원하지만 프론트엔드에서 활용하지 않음. 모든 필터링(심볼 검색, 날짜 범위)이 클라이언트 측에서 수행 **위치:** frontend/src/hooks/useOrders.ts:44-48 (파라미터 미전달), frontend/src/app/(main)/orders/page.tsx:420-440 (클라이언트 필터) **영향:** 주문 내역 1000건 + 시 전체 데이터 전송으로 초기 로드 지연 **권장:** useOrders 에 page/limit 파라미터 추가, 서버 측 심볼/날짜 필터 지원 **심각도:** High

[API-M-02] 체결 내역 — 서버 페이지네이션 미사용 (Medium, 미수정)

현상: useTradeHistory 혹은 (hooks/useOrders.ts:182-207)이 전체 체결 내역을 한번에 조회. 백엔드에서 limit/offset 지원하지만 프론트엔드에서 활용 안 함. 분석 탭에서도 전체 데이터 사용 **위치:** frontend/src/hooks/useOrders.ts:185-186 **영향:** 체결 내역 누적 시 응답 크기 선형 증가 **권장:** 커서 기반 무한 스크롤 또는 page/limit 파라미터 적용 **심각도:** Medium

[API-M-03] 뉴스 — 필터 적용 시 200건 일괄 조회 (Medium)

현상: news/page.tsx:80-81 에서 검색어나 날짜 필터가 적용되면 FILTERED_FETCH_LIMIT = 200 으로 한번에 200건을 조회한 후 클라이언트에서 필터링+페이지네이션 수행. 서버에 검색/날짜 필터 파라미터를 전달하지 않음 **위치:** frontend/src/app/(main)/news/page.tsx:78-81 **영향:** 불필요한 대량 데이터 전송, 서버 쿼리 부하 **권장:** 서버 API 에 search , publishedAfter 파라미터 추가하여 서버 측 필터링 적용 **심각도:** Medium

C. 이미지/에셋 처리 — 2건

[IMG-M-01] RichEditor — TipTap Base64 이미지 인라인 저장 (Medium, 미수정)

현상: RichEditor.tsx:125 에서 reader.readAsDataURL(file) 로 이미지를 Base64 인코딩하여 에디터 HTML 에 직접 삽입. 5MB 이미지의 Base64 인코딩 크기는 약 6.7MB로, 게시글 저장 시 DB에 대용량 문자열 저장. 해상도 4096x4096 제한이 추가되었으나 Base64 인라인 방식은 유지 **위치:** frontend/src/components/ui/RichEditor.tsx:109-127 **영향:** DB 저장 용량 비효율, 게시글 조회 시 대량 HTML 전송, 목록 페이지에서 stripHtml 호출 시에도 Base64 데이터 포함 **권장:** 이미지를 별도 스토리지(S3/uploads)에 업로드하고 URL 참조로 변경 **심각도:** Medium

[IMG-L-01] community/page.tsx stripHtml — Base64 이미지 포함 HTML 파싱 (Low)

현상: community/page.tsx:284-286 의 stripHtml 함수가 정규식으로 HTML 태그를 제거하지만, Base64 인라인 이미지가 포함된 게시글의 경우 수 MB 문자열에 대해 정규식 매칭 수행. 목록에서 다수 게시글을 렌더링할 때 성능 영향 **위치:** frontend/src/app/(main)/community/page.tsx:284-286 **영향:** Base64 이미지 포함 게시글 10건 이상 시 정규식 처리 지연 **권장:** 서버에서 미리보기용 plain text 필드를 별도로 제공하거나, Base64 제거 후 stripHtml 수행 **심각도:** Low

D. 번들/코드 분할 — 2건

[BD-M-01] 대시보드 페이지 — 516줄 단일 컴포넌트 (Medium)

현상: dashboard/page.tsx 가 516줄로 실시간 가격, WebSocket 배치, AI 분석 모달, 스포트라이트 검색, 탭 상태, 관심종목 등 모든 로직을 포함. TODO 주석(RD-M-02)으로 분리 필요성이 명시되어 있으나 미수행 **위치:** frontend/src/app/(main)/dashboard/page.tsx:1-516 **영향:** 초기 번들 크기 증가, AI 분석 모달 코드가 항상 로드됨 **권장:** AI 분석 모달, WebSocket 가격 배치 로직, 탭 상태를 별도 컴포넌트/훅으로 분리 **심각도:** Medium

[BD-L-01] lucide-react 개별 아이콘 import — 트리쉐이킹 의존 (Low, 보류)

현상: 각 컴포넌트에서 lucide-react 로부터 개별 아이콘을 직접 import. Header.tsx에서 20개+, community/page.tsx에서 17개+ 아이콘 import. Webpack/Turbopack 트리쉐이킹이 적용되므로 실질 번들 영향은 적지만, 중앙 re-export 배럴 파

일로 관리하면 import 경로 단순화 가능 위치: frontend/src/components/layout/Header.tsx:20 , frontend/src/app/(main)/community/page.tsx:28-47 영향: 빌드 시 모듈 해석 시간 미미한 증가 권장: 보류 — 트리쉐이킹이 정상 작동하는 한 실질적 성능 영향 없음 심각도: Low

E. DB 쿼리 패턴 — 2건

[DB-M-01] 관리자 체결 감사 — N+1 사용자명 조회 (Medium)

현상: order-proxy.controller.ts:160-177 에서 체결 내역 조회 후 사용자명을 enrichment하기 위해 별도 POST /users/by-ids 호출. 현재는 한번의 배치 호출이지만, 체결 내역의 userId가 다수(100+)일 경우 대량 ID 배열 전송 위치: backend/services/api-gateway/src/proxy/order-proxy.controller.ts:160-177 영향: 관리자 페이지 체결 내역 조회 시 추가 네트워크 홉 권장: order-engine DB에 사용자명 비정규화 저장 또는 Redis 캐싱으로 enrichment 최적화 심각도: Medium

[DB-L-01] 환율 캐시 — 서비스별 인메모리 중복 관리 (Low)

현상: OrderService 의 cachedExchangeRate (order.service.ts:57-58)가 서비스별 인메모리 캐시로 관리됨. 동일 환율 데이터를 api-gateway, order-engine 등 여러 서비스에서 각각 외부 API 호출하여 캐싱 위치: backend/services/order-engine/src/application/services/order.service.ts:49-58 영향: 서비스 인스턴스 수 x 외부 API 호출 중복 권장: Redis에 환율 데이터 중앙 캐싱하여 서비스 간 공유 심각도: Low

F. 캐싱/메모리 관리 — 2건

[CM-M-01] CandlestickChart — 차트 타입 변경 시 전체 시리즈 재생성 (Medium)

현상: CandlestickChart.tsx:316-401 에서 chartType 변경 시 기존 시리즈 전부 제거 후 재생성. 캔들 ↔ 라인 전환 시 메인 시리즈뿐 아니라 볼륨, SMA 3개,布林저 밴드 3개까지 총 8개 시리즈를 destroy/recreate. 이후 Effect 4에서 다시 데이터 세팅 위치: frontend/src/components/chart/CandlestickChart.tsx:316-401 영향: 차트 타입 전환 시 눈에 띄는 깜빡임과 지연 권장: 메인 시리즈만 교체하고 보조 지표 시리즈는 유지, 또는 두 시리즈를 모두 생성 후 visible 토글로 전환 심각도: Medium

[CM-L-01] 리더보드 — prevRankMap ref 누적 (Low)

현상: leaderboard/page.tsx:235 의 prevRankMap.current 가 컴포넌트 생애 동안 이전 순위 데이터를 계속 유지. 기간/정렬 필터 변경 시에도 초기화되지 않아 잘못된 순위 변동 애니메이션 발생 가능 위치: frontend/src/app/(main)/leaderboard/page.tsx:235 영향: 필터 변경 후 순위 변동 인디케이터가 실제와 다르게 표시 권장: period , sortMode 변경 시 prevRankMap.current.clear() 호출 심각도: Low

G. WebSocket/실시간 처리 — 1건

[WS-L-02] 종목 상세 페이지 — 단일 심볼에 전체 subscribe 배열 전달 (Low)

현상: asset/[symbol]/page.tsx:146 에서 useWebSocket([symbol], handlePriceUpdate) 로 단일 심볼을 구독하지만, useWebSocket 내부에서 symbols 배열이 변경될 때마다 전체 목록을 서버에 재전송 (socket.emit('subscribe', { symbols })). 종목 상세에서는 단일 심볼만 필요하므로 불필요한 재구독 발생 위치: frontend/src/hooks/useWebSocket.ts:194-196 , frontend/src/app/(main)/asset/[symbol]/page.tsx:146 영향: 종목 상세 페이지 진입 시 불필요한 subscribe 이벤트 중복 발송 권장: 단일 심볼 구독 시 diff 로직이 정확히 작동하는지 확인, 또는 subscribe 시 이전 구독과 동일하면 스킵 심각도: Low

종합 평가

7차 성능 감사에서 총 17건의 이슈를 발견하였습니다. 6차 대비 FLIP 애니메이션 배치 처리(FR-H-02), 채팅방 폴링 최적화(WS-L-01) 등이 수정되었으나, 커뮤니티 페이지 거대 컴포넌트(PF-M-01), 주문/체결 서버 페이지네이션 미활용(API-M-01/02), TipTap Base64 이미지(IMG-M-01)는 여전히 미수정 상태입니다.

가장 시급한 개선 항목은:

1. 주문 내역 서버 페이지네이션 적용 (API-M-01) — 데이터 누적 시 성능 저하 직결
2. AI 분석 뉴스 과잉 조회 제거 (PF-H-01) — 불필요한 네트워크/서버 부하
3. Base64 이미지 외부 스토리지 전환 (IMG-M-01) — DB 용량 및 전송 효율

전체적으로 프론트엔드의 데이터 패칭 전략이 "전체 조회 + 클라이언트 필터" 패턴에 의존하고 있어, 데이터 누적 시 선형적 성능 저하가 예상됩니다. 서버 측 필터/페이지네이션 활용을 우선적으로 개선해야 합니다.